Project Description

BrokBuddy is a smart dog food system that helps dog owners monitor the amount of food that is still available. In the complete concept, a weight sensor measures the food supply and the owner receives a warning when the amount of food becomes too low. 26-10-07 IoT BROKBUDDY

This manual focuses on a small technical prototype of BrokBuddy using a NodeMCU ESP8266. Since no external weight sensor is available, the food level is simulated in the Arduino code.

The ESP8266 connects to a WiFi network so it can communicate with a Telegram bot. The program checks the simulated food level. When the food level drops below the set limit, the built-in LED of the ESP8266 turns on and a warning message is automatically sent to the owner through Telegram.

The prototype consists of:

Simulated food level > ESP8266 > WiFi connection > low-food detection > built-in LED > Telegram notification

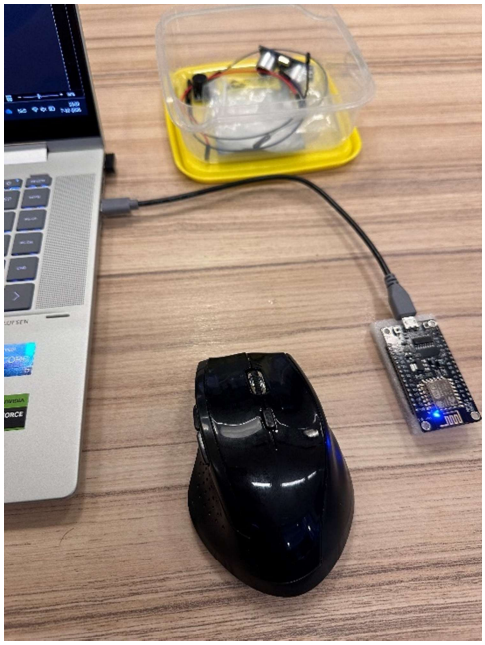
The goal of this manual is to show step by step how to build, test and troubleshoot this prototype.

Step 1: Connect the NodeMCU ESP8266 to your computer

Start by connecting the **NodeMCU ESP8266** to your computer with a USB cable.

1. Plug the USB cable into the NodeMCU ESP8266.
2. Connect the other end of the cable to a USB port on your computer.
3. Check the NodeMCU. When it receives power, the small blue LED may briefly flash.
4. Leave the NodeMCU connected to the computer.

At this point, the board is powered and ready to be detected by the Arduino IDE.

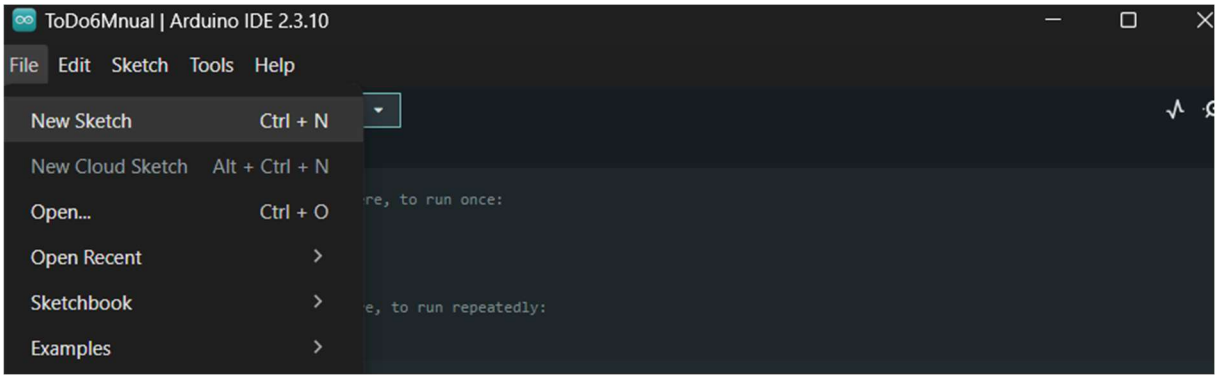


Checkpoint

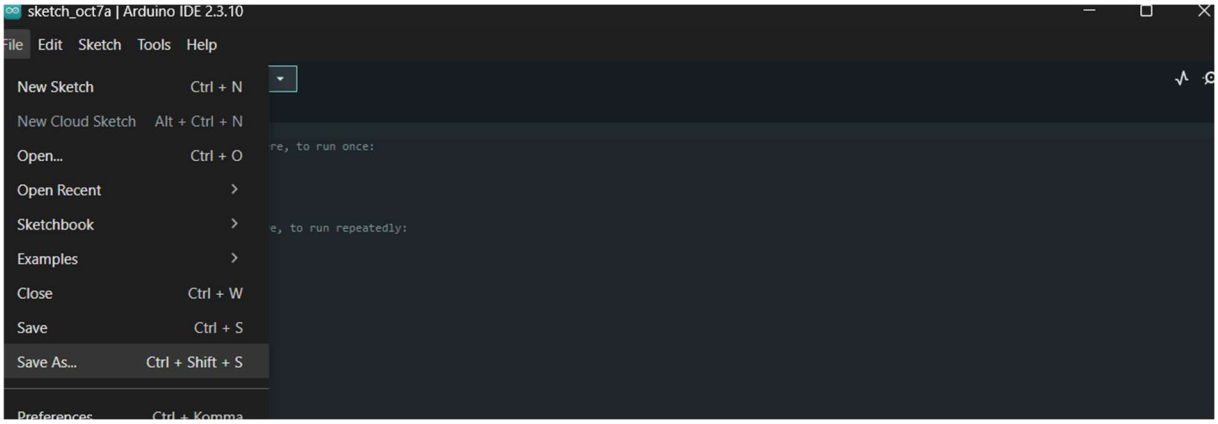
The NodeMCU is connected to your computer and receives power. A short flash from the blue LED when connecting the board is normal. The course material also uses this as a basic check that the board is receiving power.

Step 2: Create a new Arduino sketch

Open the Arduino IDE on your computer. Click **File > New Sketch** to create a new Arduino file.



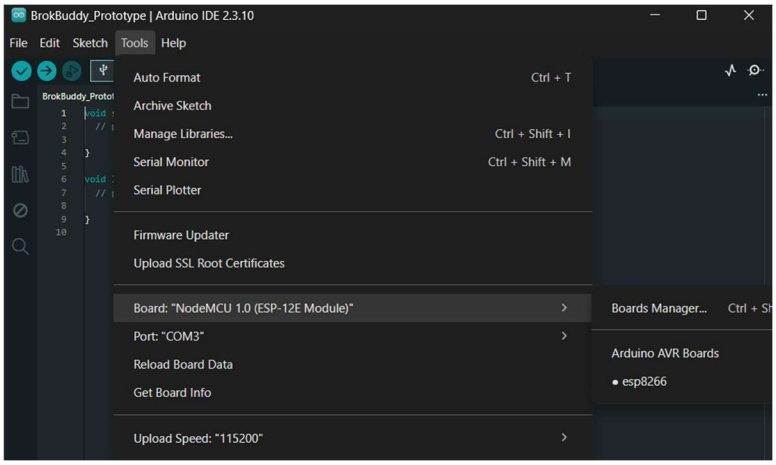
Then click **File > Save As** and save the sketch with the name **BrokBuddy_Prototype**.



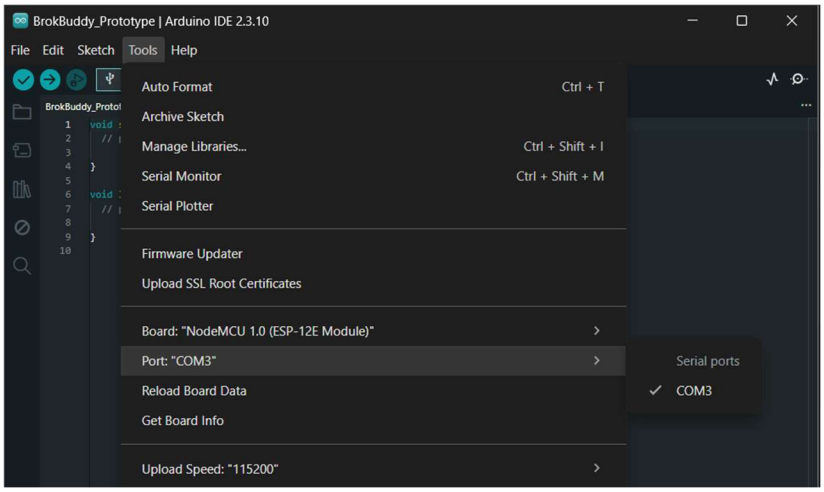
Now the new Arduino sketch is now ready to use.

Step 3 - Select the correct board and COM port

Make sure the NodeMCU ESP8266 is connected to your computer. In the Arduino IDE, go to **Tools > Board** and select **NodeMCU 1.0 (ESP-12E Module)**.



Then go to **Tools > Port** and select the COM port that belongs to the NodeMCU.

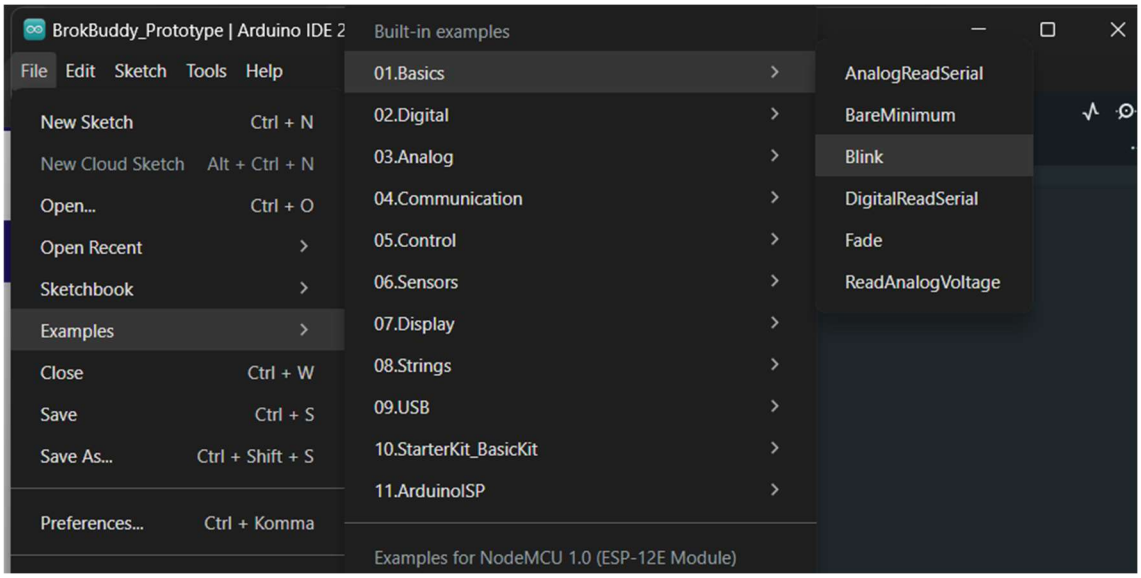


Checkpoint
The selected board is **NodeMCU 1.0 (ESP-12E Module)** and a COM port is visible and selected.

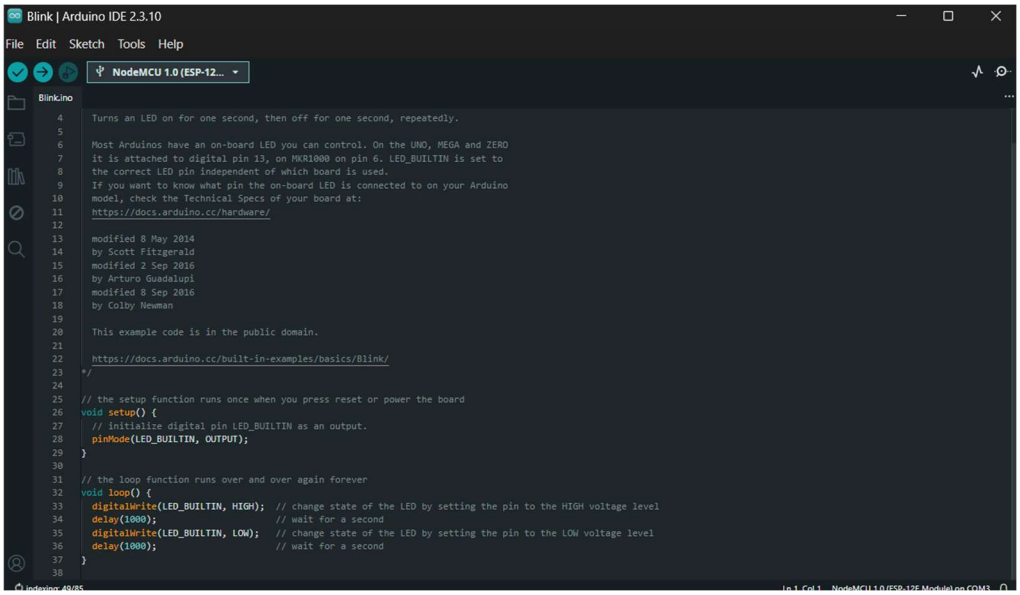
The correct NodeMCU board and COM port are now selected. The Arduino IDE is ready to upload code to the ESP8266.

Step 4 - Test the built-in LED

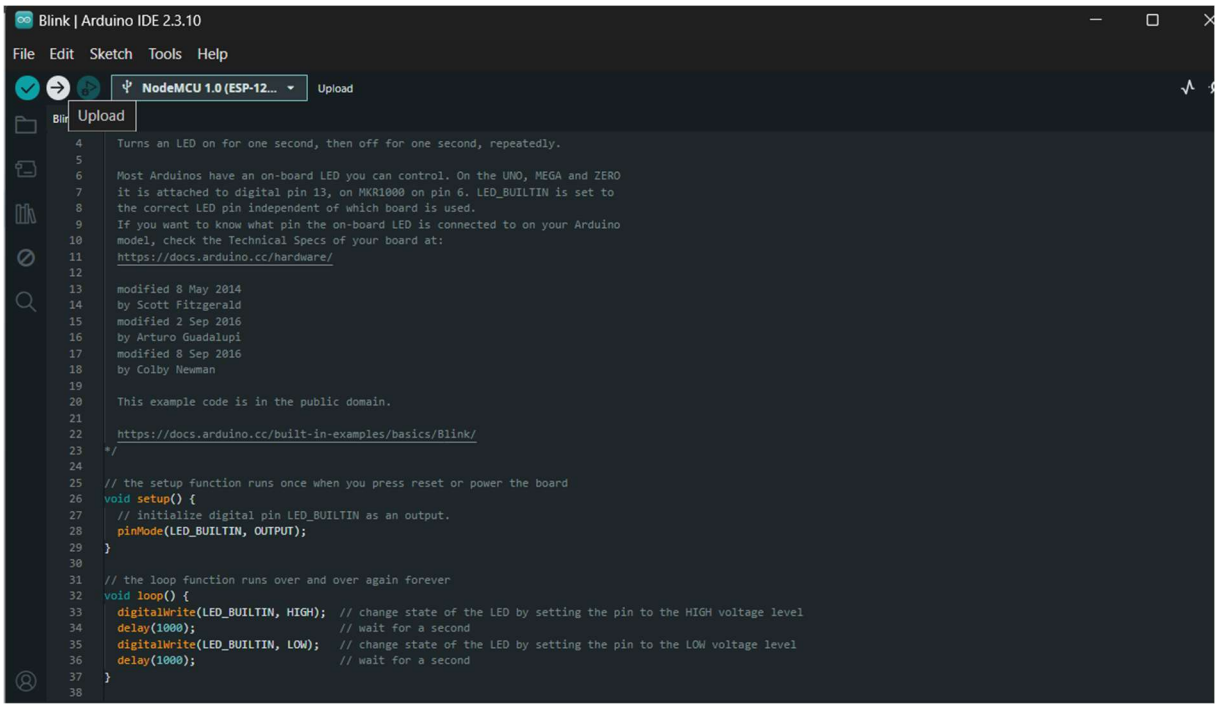
To check if the NodeMCU is working correctly, open the Arduino IDE and go to **File > Examples > 01.Basics > Blink**.



The Blink example will open in a new window.



Make sure the correct NodeMCU board and COM port are still selected. Then click the **Upload** button to upload the code to the ESP8266.



After the upload is complete, the built-in LED on the NodeMCU should start blinking. This confirms that the board can receive and run Arduino code.

If the upload is successful, the output should show that the sketch was uploaded without errors.



Checkpoint

The built-in LED turns on and off automatically after the Blink sketch has been uploaded.

Step 5 - Open and test the Serial Monitor

Open your saved **BrokBuddy_Prototype** sketch again.

Add this simple code:

```
void setup() {
 Serial.begin(9600);
 Serial.println("BrokBuddy started");
}

void loop() {
}
```



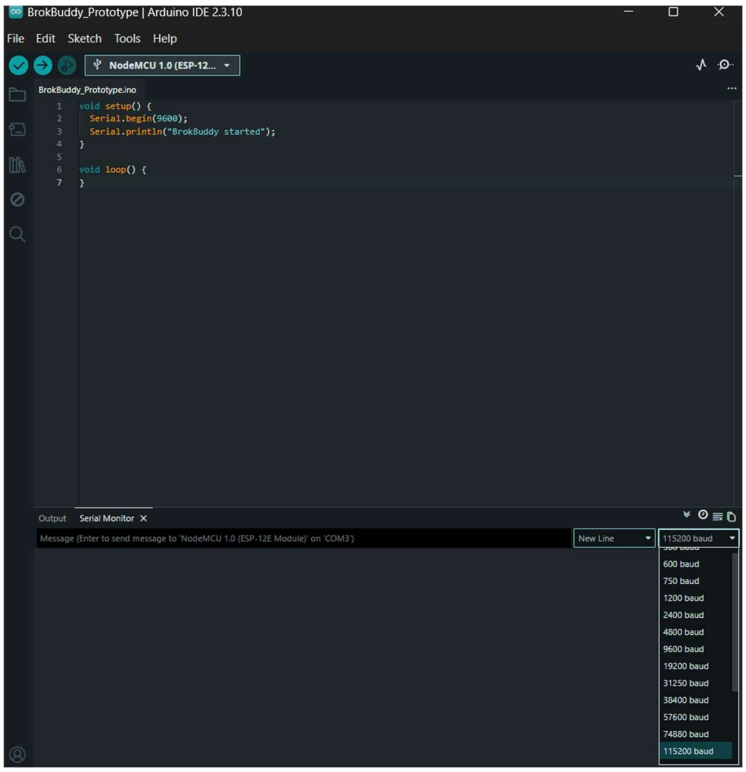
Possible problem - Nothing appears in the Serial Monitor

Problem:
Nothing appears in the Serial Monitor or the text is unreadable.



Cause:
The baud rate in the Serial Monitor does not match the baud rate used in the Arduino code.

Solution:
Set the baud rate in the Serial Monitor to **9600 baud**, because the code uses:

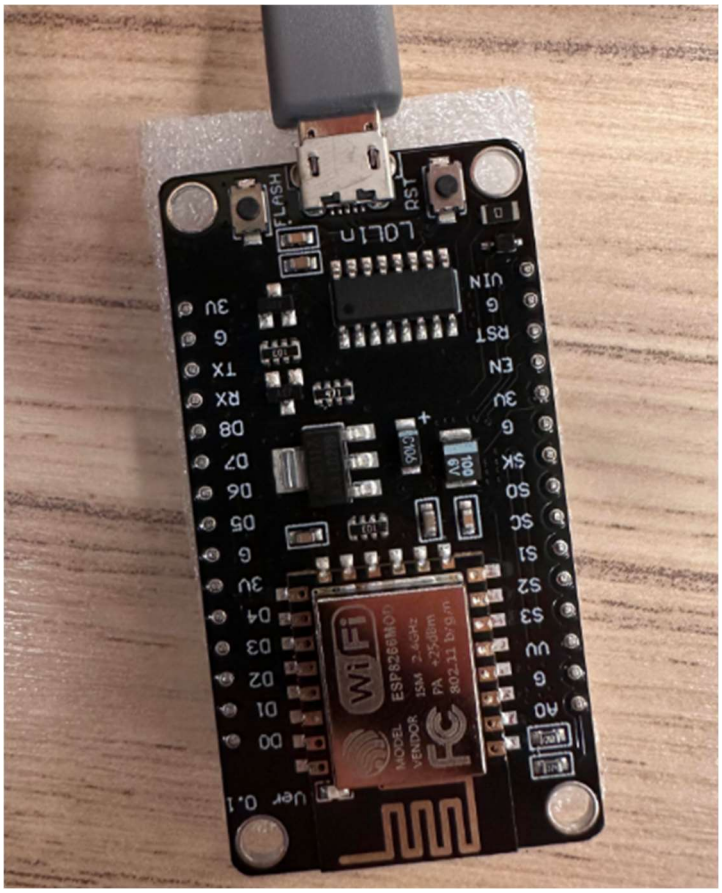


Possible problem - No message appears in the Serial Monitor



If no message appears after setting the Serial Monitor to **9600 baud**, press the **RST button** on the NodeMCU once.

This restarts the program and the message should then appear in the Serial Monitor.



Possible problem - Strange characters appear in the Serial Monitor

After pressing the reset button, some unreadable characters may appear before the normal message.

This is normal for the ESP8266 during startup. The important part is that the correct message appears afterwards:



Step 6 - Connect the NodeMCU to WiFi

The NodeMCU needs an internet connection to send Telegram messages later. Add the WiFi library and enter the name and password of your WiFi network.

The following GitHub example is used as a reference for the WiFi and Telegram setup:

https://github.com/witnessmenow/Universal-Arduino-Telegram-Bot/blob/master/examples/ESP8266/FlashLED/FlashLED.ino

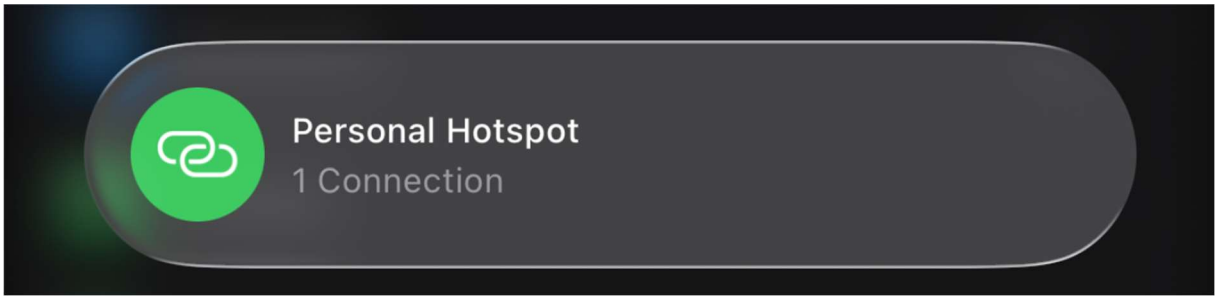
In the next step, the WiFi settings from this example are added to the BrokBuddy prototype.

```
19    | Twitter: https://twitter.com/witnessmenow
20    *******************************************************************/
21
22    #include <ESP8266WiFi.h>
23    #include <WiFiClientSecure.h>
24    #include <UniversalTelegramBot.h>
25
26    // Wifi network station credentials
27    #define WIFI_SSID "YOUR_SSID"
28    #define WIFI_PASSWORD "YOUR_PASSWORD"
29    // Telegram BOT Token (Get from Botfather)
30    #define BOT_TOKEN "XXXXXXXXX:XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX"
31
32    const unsigned long BOT_MTBS = 1000; // mean time between scan messages
```

Replace YOUR_SSID with the name of the WiFi network or mobile hotspot you want to use. Replace YOUR_PASSWORD with the password of that WiFi network or hotspot.

After adding the correct hotspot details, upload the code to the NodeMCU.

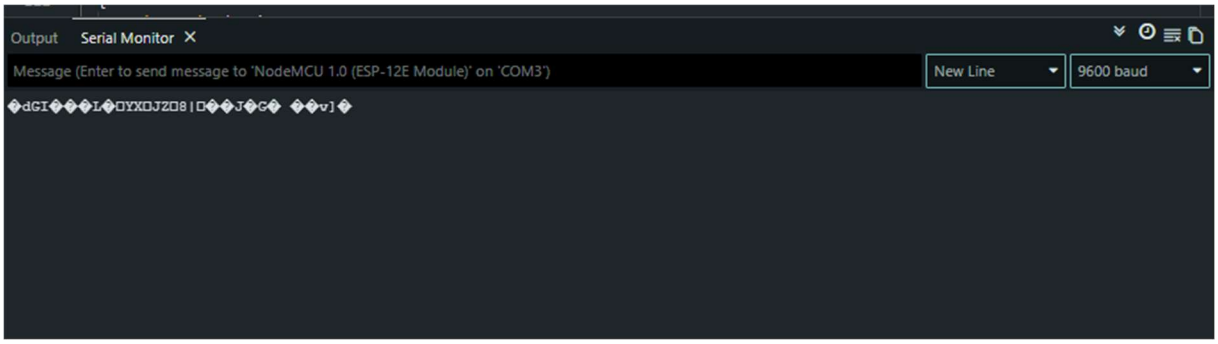
Open the Serial Monitor and set it to the same baud rate as used in the code. The NodeMCU should now try to connect to the hotspot.



Checkpoint

When the connection is successful, the Serial Monitor should show that the NodeMCU is connected to WiFi.

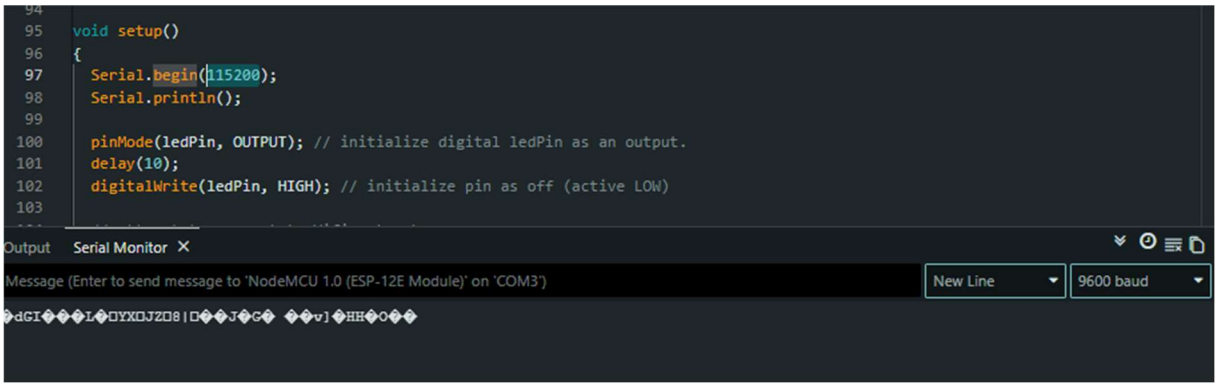
If it keeps trying to connect or cannot find the hotspot, we use that as the error for this step and document the cause and solution.



Possible problem - Unreadable output in the Serial Monitor

Problem:
The Serial Monitor shows unreadable characters instead of clear connection information.

Cause:
The baud rate in the Serial Monitor does not match the baud rate used in the Arduino code.



Solution:
Check the baud rate in the code, for example:’’Serial.begin(9600);’’ in line 97



After matching the baud rate in the code and the Serial Monitor, restart the NodeMCU and check whether the WiFi connection information is displayed correctly.

Step 7 - Install Telegram

First install the **Telegram** app on your phone or computer.

1. Open the **App Store**, **Google Play Store**, or the official Telegram website.
2. Search for **Telegram Messenger**.
3. Install the application.
4. Open Telegram and sign in with your phone number.
5. Complete the verification process.

Checkpoint

Telegram is installed and you can open the app and send messages.

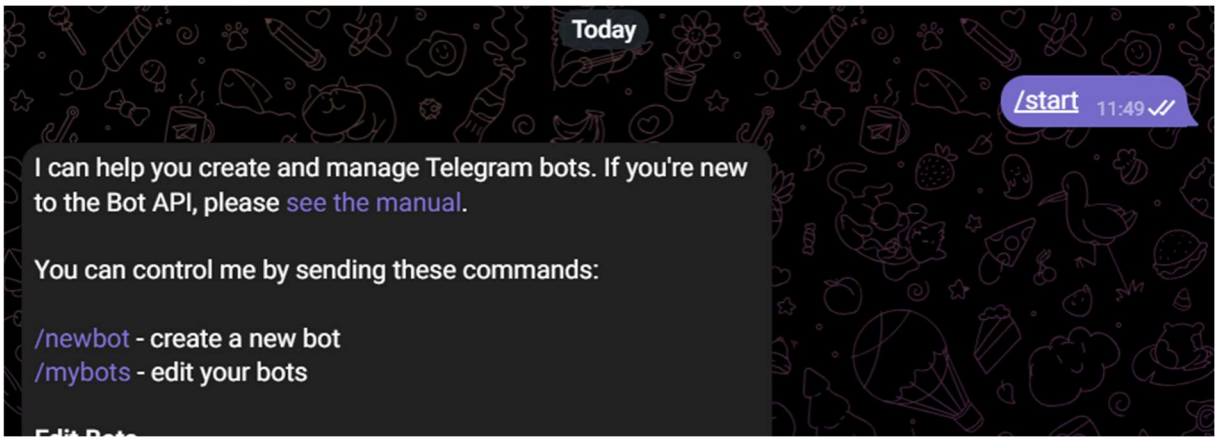
Step 8 - Create the BrokBuddy Telegram bot

Open Telegram and search for **BotFather**.

First send:

/start

BotFather will show the available commands.



Then send:

/newbot

BotFather will ask you to choose a name for the bot. Enter:

BrokBuddy

Next, choose a username that ends with bot, for example:

BrokBuddy_IoT_bot

After the bot has been created, BotFather will send you a **bot token**. Save this token because you will need it later in the Arduino code.

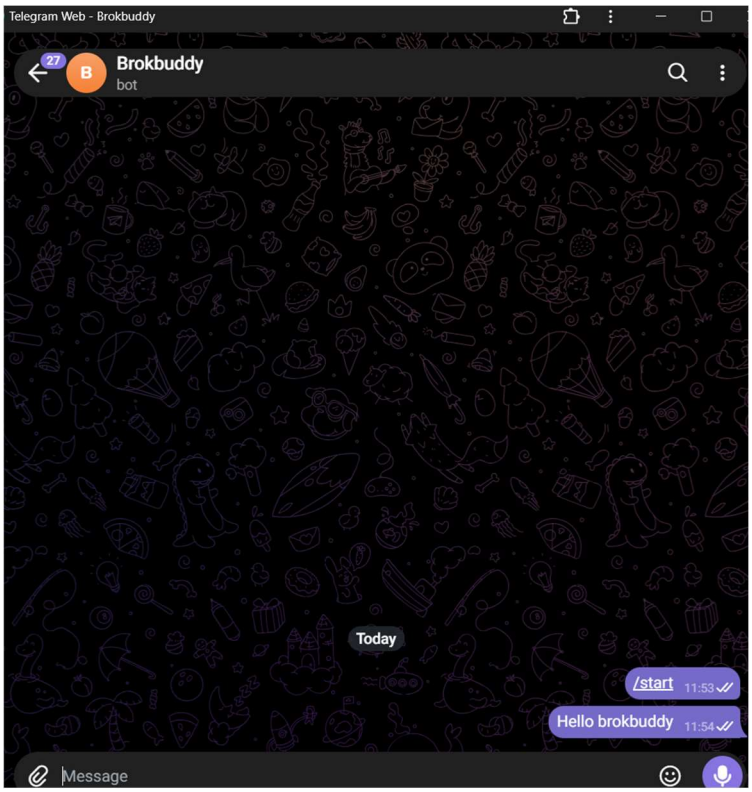
Step 9 - Start the BrokBuddy bot

Open your bot in this case: **BrokBuddy_IoT_bot** in Telegram and press **Start**.

Send a test message to the bot, for example:

Hello BrokBuddy

This message is needed so Telegram can create an update that contains your Chat ID.



After sending a message to the BrokBuddy bot, the connection can be tested through the Telegram Bot API.

Open a web browser and enter:

https://api.telegram.org/botYOUR_BOT_TOKEN/getMe

Replace YOUR_BOT_TOKEN with the token received from BotFather.

If the token is correct, the browser will return information about the bot and show:

"ok":true

This confirms that the Telegram bot has been created correctly and that the bot token is working.

Step 10 - Test the Bot Token

Before adding the Bot Token to the Arduino code, first check if the token works correctly.Open a web browser and enter:

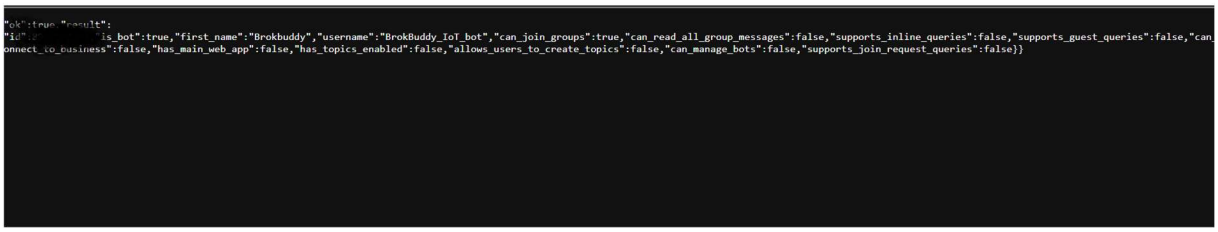
https://api.telegram.org/botYOUR_BOT_TOKEN/getMe

Replace YOUR_BOT_TOKEN with the full token received from BotFather.

If the token is correct, the browser will display information about the BrokBuddy bot. Look **for:**

"ok":true

This confirms that the Bot Token is valid and that Telegram can recognize the BrokBuddy bot.



The browser will now display information about the messages sent to the bot. Look for:

"chat":{"id":123456789

The number after "id": is your **Chat ID.**

Copy and save this number. It will be needed in the Arduino code later.

Checkpoint

The Bot Token works correctly and the Chat ID has been found and saved.

Step 11 - Add the Bot Token and Chat ID to the Arduino code

Return to the Arduino IDE and open the **BrokBuddy_Prototype** code.

Find the section where the WiFi information and Telegram Bot Token are stored. Replace the existing Bot Token with the **full token** received from BotFather.

Then add a new line underneath for the Chat ID:

```
#include <WiFiClientSecure.h>
#include <UniversalTelegramBot.h>

// Wifi network station credentials
#define WIFI_SSID "PhilipIphone"
#define WIFI_PASSWORD "schoolopdr123"
// Telegram BOT Token (Get from Botfather)
#define BOT_TOKEN "XXXXXXXXX:XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX"

const unsigned long BOT_MTBS = 1000; // mean time between scan messages

X509List cert(TELEGRAM_CERTIFICATE_ROOT);
WiFiClientSecure secured_client;
UniversalTelegramBot bot(BOT_TOKEN, secured_client);
unsigned long bot_lasttime; // last time messages' scan has been done
```

Replace YOUR_FULL_BOT_TOKEN with your complete Bot Token and replace YOUR_CHAT_ID with the Chat ID found in Step 10.

```
    Written by Brian Lough
    YouTube: https://www.youtube.com/brianlough
    Tindie: https://www.tindie.com/stores/brianlough/
    Twitter: https://twitter.com/witnessmenow
 *******************************************************************/

#include <ESP8266WiFi.h>
#include <WiFiClientSecure.h>
#include <UniversalTelegramBot.h>

// Wifi network station credentials
#define WIFI_SSID "PhilipIphone"
#define WIFI_PASSWORD "schoolopdr123"
// Telegram BOT Token (Get from Botfather)
#define BOT_TOKEN "XXXXXXXXX:XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX"
#define CHAT_ID "YOUR_CHAT_ID"

const unsigned long BOT_MTBS = 1000; // mean time between scan messages

X509List cert(TELEGRAM_CERTIFICATE_ROOT);
WiFiClientSecure secured_client;
UniversalTelegramBot bot(BOT_TOKEN, secured_client);
unsigned long bot_lasttime; // last time messages' scan has been done

const int ledPin = LED_BUILTIN;
int ledStatus = 0;
```

Possible problem - Telegram does not work after adding the Chat ID

Problem:
The NodeMCU connects to WiFi, but the Telegram bot does not send messages.

Possible cause:
The Chat ID or Bot Token is not written correctly in the Arduino code.

Solution:
Check that both values are placed between quotation marks:

#define BOT_TOKEN "YOUR_FULL_BOT_TOKEN"

#define CHAT_ID "YOUR_CHAT_ID"

Checkpoint

The WiFi details, Bot Token and Chat ID are now added to the Arduino code. The NodeMCU has the information needed to connect to WiFi and communicate with the correct Telegram chat.

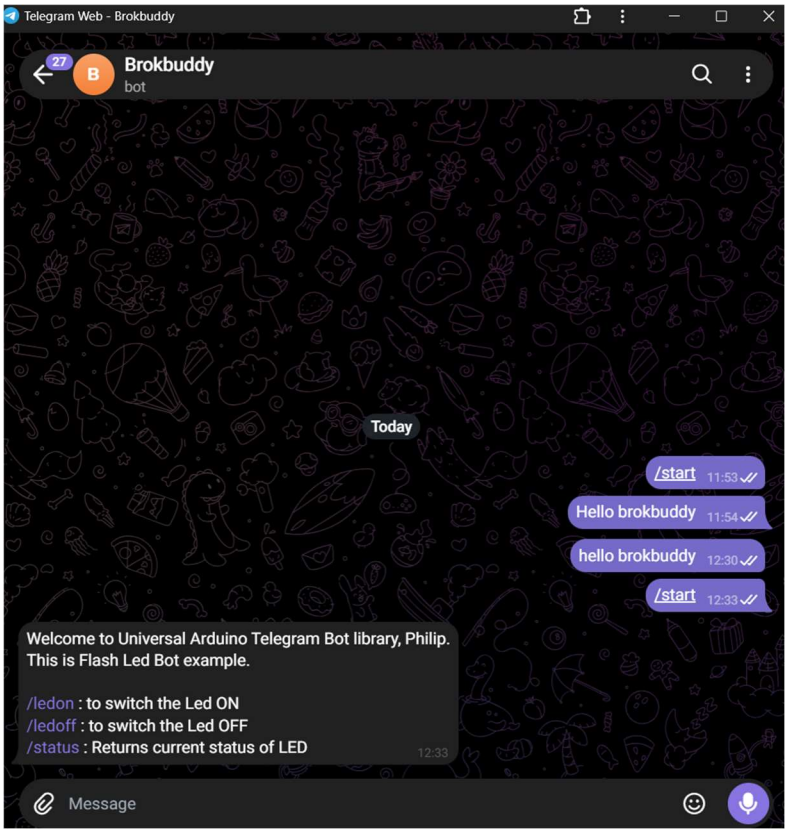
If all WiFi and Telegram details are entered correctly, click the **Upload** button to upload the code to the NodeMCU.

Result:
If the Bot Token and Chat ID are correct, the NodeMCU can communicate with the BrokBuddy Telegram bot.

Step 13 - Test the Telegram bot

Open the **BrokBuddy bot** in Telegram and send the command:

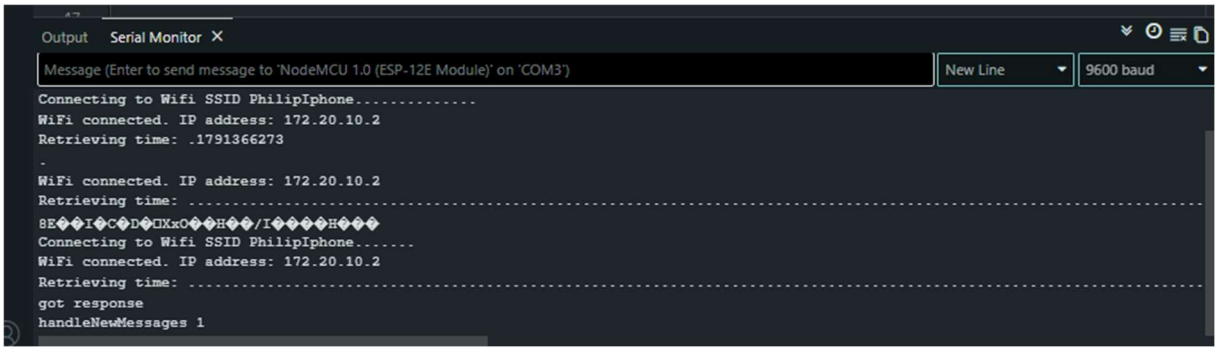
/start



Wait a few seconds for the NodeMCU to receive the message.

If the connection is working correctly, the bot should respond in Telegram.

At the same time, check the **Serial Monitor** in the Arduino IDE. It should show that the NodeMCU is still connected to WiFi and is receiving Telegram messages.



Checkpoint

The BrokBuddy bot responds to /start. This confirms that the NodeMCU, WiFi connection and Telegram bot are working together.

Possible problem - The bot does not respond

If the bot does not respond, first check if the Serial Monitor still shows:

WiFi connected

Also check that the **Bot Token** and **Chat ID** are entered correctly in the Arduino code.

If necessary, press the **RST button** on the NodeMCU and try /start again.

Step 14 - Test the built-in LED with Telegram

Now test whether the Telegram bot can control the built-in LED on the NodeMCU.

Send the following command to the BrokBuddy bot:

/ledon

The built-in LED on the NodeMCU should turn on.

Then send:

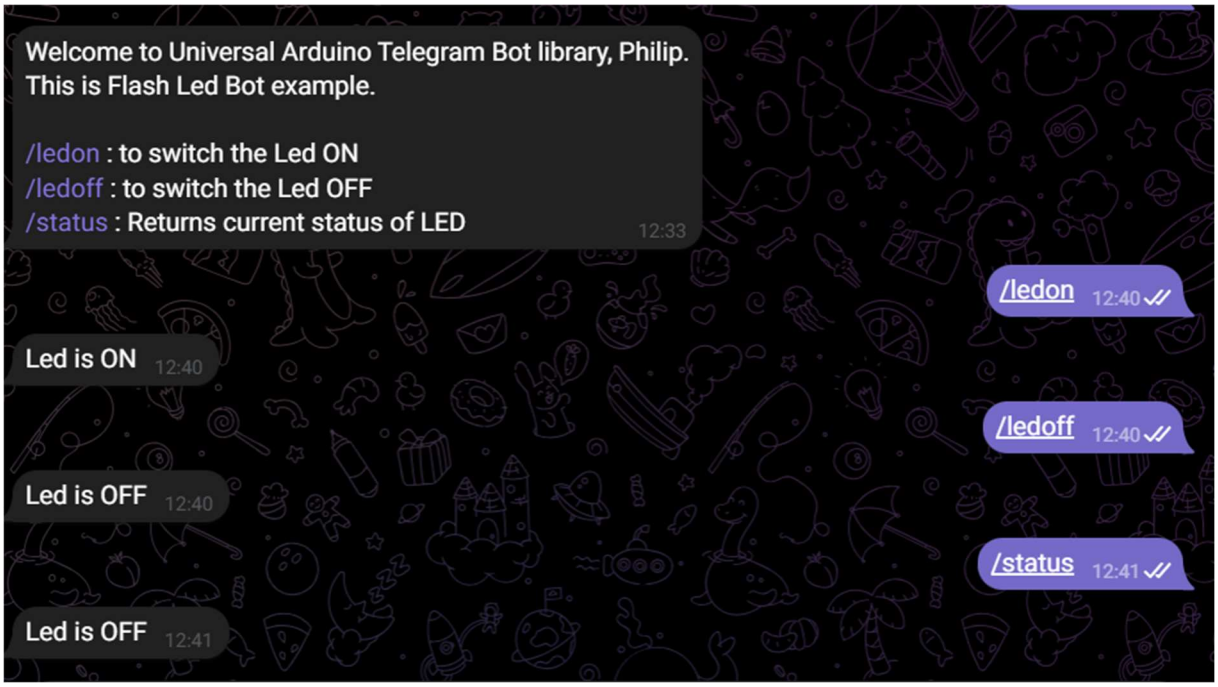
/ledoff

The built-in LED should turn off again.

You can also send:

/status

The bot should return the current status of the LED.



Result

The BrokBuddy bot successfully controls the built-in LED on the NodeMCU. The /ledon command switches the LED on, /ledoff switches it off, and /status returns the current LED status.

Checkpoint

The NodeMCU receives Telegram commands and responds correctly. This confirms that the Telegram bot can control a physical output on the ESP8266.

Step 15 - Add a simulated food level

The BrokBuddy prototype does not use a real weight sensor yet. Therefore, the amount of dog food will be simulated with a value in the Arduino code.

In the Arduino code, go to the section with the global variables. This is the part near the top of the code where ledPin and ledStatus are already defined.

```
#define CHAT_ID "id":8956593164"

const unsigned long BOT_MTBS = 1000; // mean time between scan messages

X509List cert(TELEGRAM_CERTIFICATE_ROOT);
WiFiClientSecure secured_client;
UniversalTelegramBot bot(BOT_TOKEN, secured_client);
unsigned long bot_lasttime; // last time messages' scan has been done

const int ledPin = LED_BUILTIN;
int ledStatus = 0;

void handleNewMessages(int numNewMessages)
{
  Serial.print("handleNewMessages ");
  Serial.println(numNewMessages);

  for (int i = 0; i < numNewMessages; i++)
  {
    String chat_id = bot.messages[i].chat_id;
    String text = bot.messages[i].text;

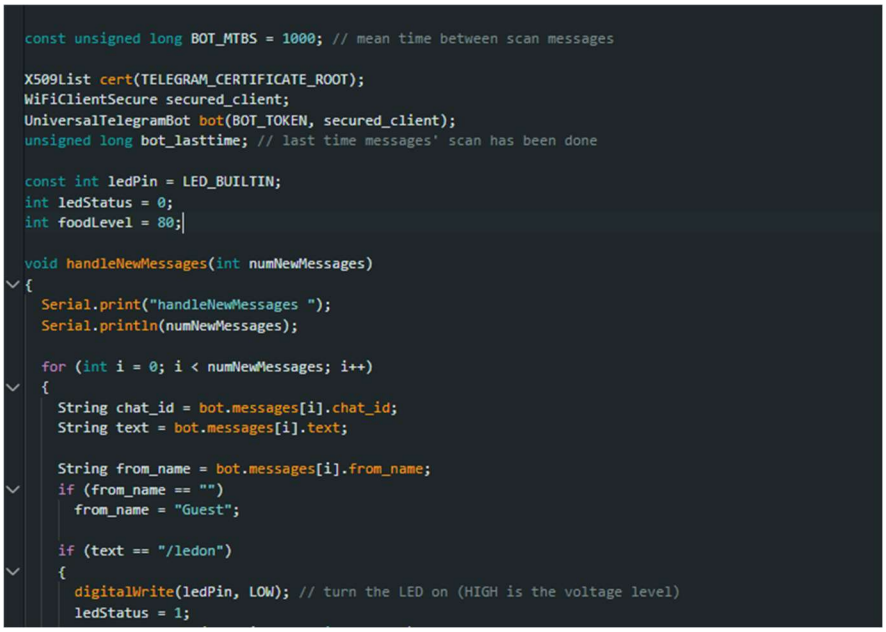
    String from_name = bot.messages[i].from_name;
    if (from_name == "")
```

Add the new variable directly underneath:

```
const int ledPin = LED_BUILTIN;

int ledStatus = 0;

int foodLevel = 80;
```



```
const unsigned long BOT_MTBS = 1000; // mean time between scan messages

X509List cert(TELEGRAM_CERTIFICATE_ROOT);
WiFiClientSecure secured_client;
UniversalTelegramBot bot(BOT_TOKEN, secured_client);
unsigned long bot_lasttime; // last time messages' scan has been done

const int ledPin = LED_BUILTIN;
int ledStatus = 0;
int foodLevel = 80;

void handleNewMessages(int numNewMessages)
{
  Serial.print("handleNewMessages ");
  Serial.println(numNewMessages);

  for (int i = 0; i < numNewMessages; i++)
  {
    String chat_id = bot.messages[i].chat_id;
    String text = bot.messages[i].text;

    String from_name = bot.messages[i].from_name;
    if (from_name == "")
      from_name = "Guest";

    if (text == "/ledon")
    {
      digitalWrite(ledPin, LOW); // turn the LED on (HIGH is the voltage level)
      ledStatus = 1;
```

Possible problem - foodLevel is placed inside setup()

Problem:
The foodLevel variable is added inside the setup() function.

```
void setup() {
  int foodLevel = 80;
}
```

Cause:
A variable created inside setup() can only be used inside that function.

Solution:
Move foodLevel outside of setup() and place it near the other global variables:

```
const int ledPin = LED_BUILTIN;

int ledStatus = 0;

int foodLevel = 80;
```

Result:
The foodLevel variable can now be used throughout the entire BrokBuddy program.

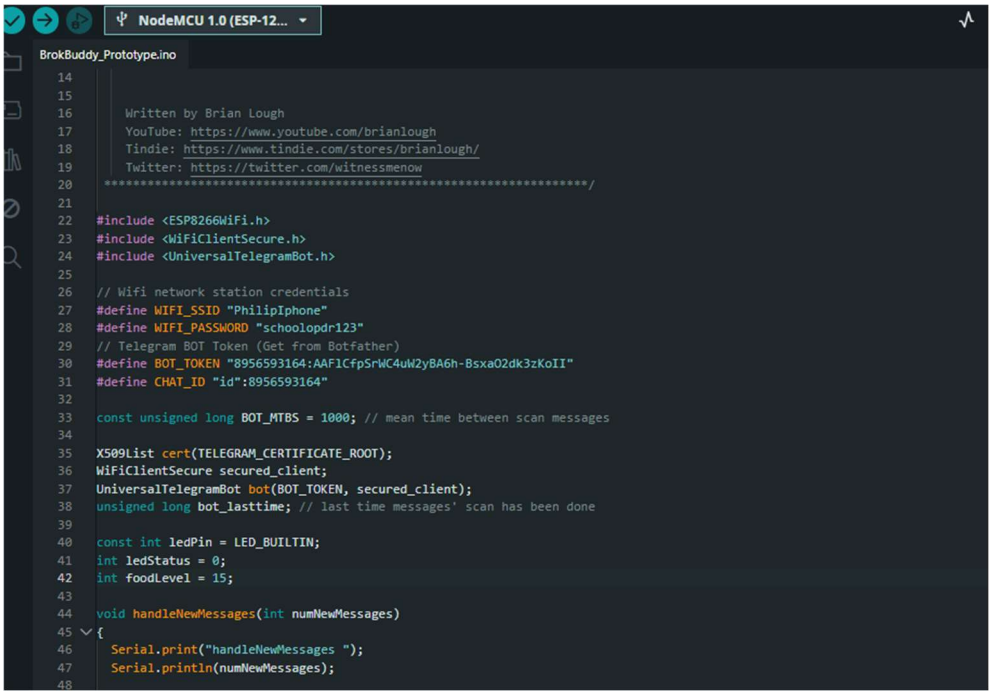
Dan is deze fout netjes compleet voor je rubric.

After adding the correct foodLevel variable, click the **Upload** button to upload the updated code to the NodeMCU.

Step 16 - Add the low food detection

Now add a condition inside the loop() function that checks whether the simulated food level is low.

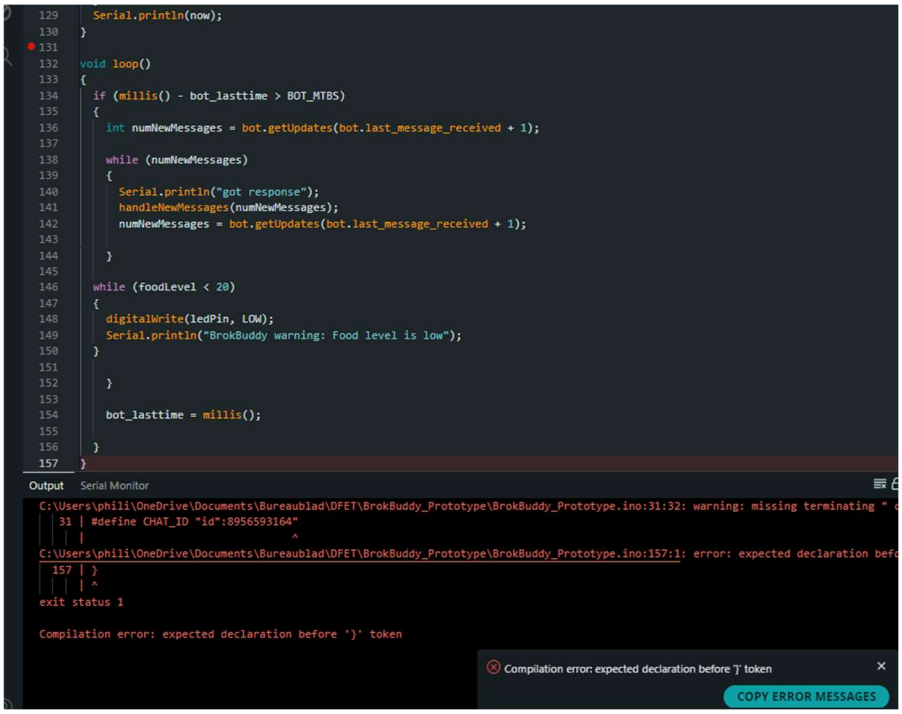
First, change the simulated food level near the top of the Arduino code from: int foodLevel = 80;
To: int foodLevel = 15;



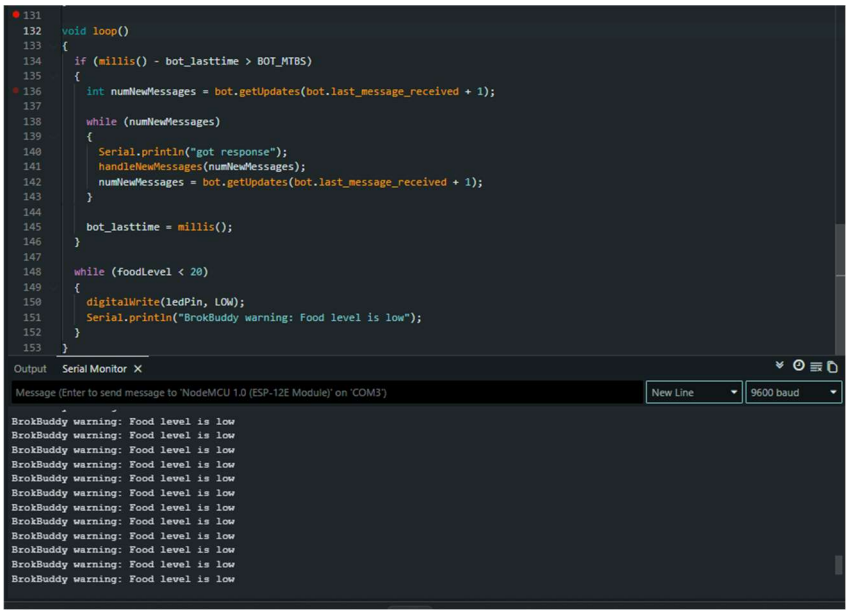
This simulates that only 15 percent of the dog food is left.

Next, scroll down to the void loop() function. Add the following code underneath the existing Telegram code, but still inside the loop() function:

```
while (foodLevel < 20) {
  digitalWrite(ledPin, LOW);
  Serial.println("BrokBuddy warning: Food level is low");
}
```



Upload the code to the NodeMCU and open the Serial Monitor to check how BrokBuddy reacts when the simulated food level is set to 15 percent.



Problem - The warning keeps repeating and Telegram stops responding

When foodLevel is set to 15, the Serial Monitor keeps showing:

‘’BrokBuddy warning: Food level is low’’

As long as foodLevel stays below 20, this loop never ends.

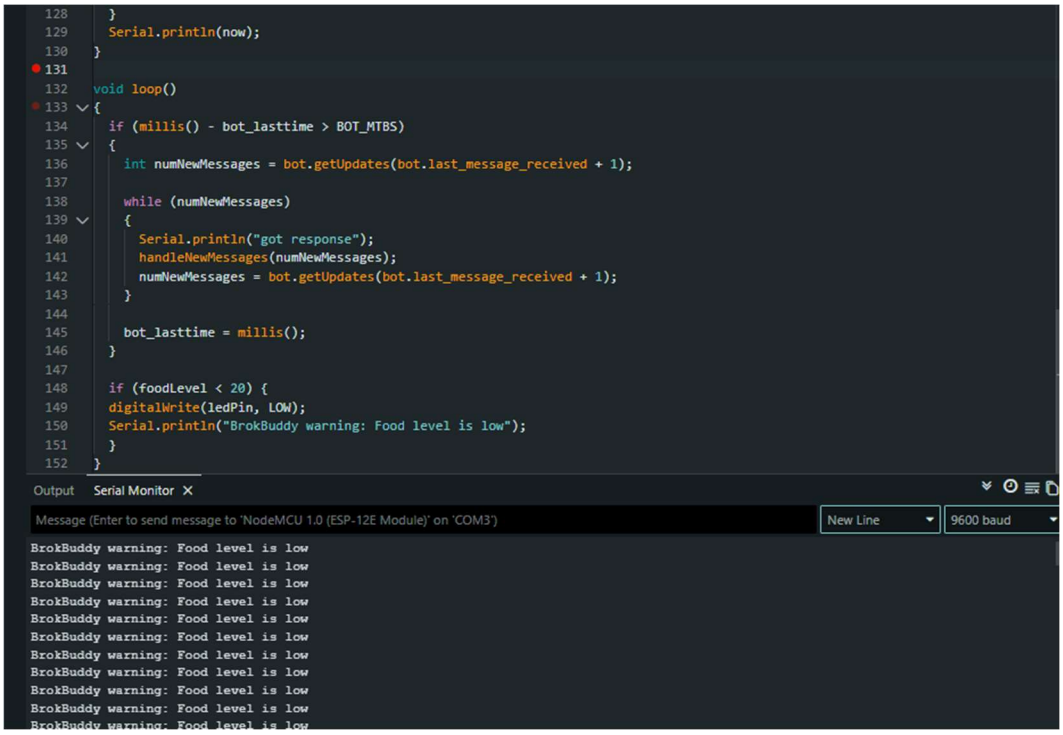
Fix

Replace:

```
while (foodLevel < 20) {
 digitalWrite(ledPin, LOW);
 Serial.println("BrokBuddy warning: Food level is low");
}
```

with:

```
if (foodLevel < 20) {
 digitalWrite(ledPin, LOW);
 Serial.println("BrokBuddy warning: Food level is low");
}
```



Problem - The warning still keeps repeating

The while loop was replaced with an if statement, so the program no longer gets stuck. However, the warning is still printed continuously in the Serial Monitor.

Cause:
The Arduino loop() function runs over and over again. Because foodLevel remains below 20, the condition is true every time the loop starts again.

Final fix - Prevent the warning from repeating

The program needs to remember whether the low-food warning has already been activated. A boolean variable is used for this.

First, go to the global variables near the top of the Arduino code. Directly underneath:
int foodLevel = 15;

add: bool lowFoodWarningSent = false;

The variables should now look like this:



Next, go back to the loop() function and replace the previous low-food code with:

```
if (foodLevel < 20 && lowFoodWarningSent == false)
{
 digitalWrite(ledPin, LOW);
 Serial.println("BrokBuddy warning: Food level is low");
 lowFoodWarningSent = true;
}

if (foodLevel >= 20)
{
 digitalWrite(ledPin, HIGH);
 lowFoodWarningSent = false;
}
```

```
}

void loop()
{
  if (millis() - bot_lasttime > BOT_MTBS)
  {
    int numNewMessages = bot.getUpdates(bot.last_message_received + 1);

    while (numNewMessages)
    {
      Serial.println("got response");
      handleNewMessages(numNewMessages);
      numNewMessages = bot.getUpdates(bot.last_message_received + 1);
    }

    bot_lasttime = millis();
  }

if (foodLevel < 20 && lowFoodWarningSent == false)
{
  digitalWrite(ledPin, LOW);
  Serial.println("BrokBuddy warning: Food level is low");
  lowFoodWarningSent = true;
}

if (foodLevel >= 20)
{
  digitalWrite(ledPin, HIGH);
  lowFoodWarningSent = false;
}
}
```

Final fix

A boolean variable is used to remember whether the low-food warning has already been activated. When the food level drops below 20 percent, the warning is triggered once and lowFoodWarningSent is set to true.

When the food level rises back to 20 percent or higher, the warning status is reset.

Result

The low-food warning is now shown only once. BrokBuddy can continue running normally and the warning can be triggered again after the food level has been refilled.



Step 17 - Send the low food warning to Telegram

Now BrokBuddy will send the low-food warning automatically to Telegram.

Scroll to the bottom of the Arduino code and find the low-food section inside void loop(). In the current code, this is around **line 149 to 153**.

Find this line: Serial.println("BrokBuddy warning: Food level is low");

Directly underneath this line, add:

bot.sendMessage(CHAT_ID, "BrokBuddy warning: Food level is low. Please refill the container.", "");

The complete section should now look like this:

```
144     }
145
146     bot_lasttime = millis();
147   }
148
149   if (foodLevel < 20 && lowFoodWarningSent == false)
150   {
151     digitalWrite(ledPin, LOW);
152     Serial.println("BrokBuddy warning: Food level is low");
153     bot.sendMessage(CHAT_ID, "BrokBuddy warning: Food level is low. Please refill the container.", "");
154     lowFoodWarningSent = true;
155   }
156
157   if (foodLevel >= 20)
158   {
159     digitalWrite(ledPin, HIGH);
160     lowFoodWarningSent = false;
```

After upload

```
48    Serial.println(numNewMessages);
49
Output  Serial Monitor
                    ^~~~~~~
C:\Users\phili\OneDrive\Documents\Bureaublad\DFET\BrokBuddy_Prototype\BrokBuddy_Prototype.ino: In function 'void loop()':
C:\Users\phili\OneDrive\Documents\Bureaublad\DFET\BrokBuddy_Prototype\BrokBuddy_Prototype.ino:31:21: error: expected ')' before ':' to
   31 | #define CHAT_ID "id":8956593164"
      |                     ^
C:\Users\phili\OneDrive\Documents\Bureaublad\DFET\BrokBuddy_Prototype\BrokBuddy_Prototype.ino:153:19: note: in expansion of macro 'CHA
  153 |   bot.sendMessage(CHAT_ID, "BrokBuddy warning: Food level is low. Please refill the container.", "");
      |                   ^~~~~~~
C:\Users\phili\OneDrive\Documents\Bureaublad\DFET\BrokBuddy_Prototype\BrokBuddy_Prototype.ino:153:18: note: to match this '('
  153 |   bot.sendMessage(CHAT_ID, "BrokBuddy warning: Food level is low. Please refill the container.", "");
      |                  ^
exit status 1

Compilation error: missing terminating " character
```

Compilation error: missing terminating " character

COPY ERROR MESSAGES

Problem - The Chat ID was written incorrectly

After adding the Telegram warning, the Arduino code did not compile and showed an error such as:

missing terminating " character

The problem was caused by the way the Chat ID was written in the code.

Incorrect: #define CHAT_ID "id":12345678"

Correct: #define CHAT_ID "123456789"

Result after fixing the Chat ID syntax

After correcting the Chat ID format, the Arduino code compiled and uploaded successfully.

The NodeMCU connected to WiFi and the low-food detection also worked correctly. The Serial Monitor showed:

BrokBuddy warning: Food level is low

However, no automatic warning was received in Telegram.

Problem - No Telegram notification is received

The low-food detection works correctly and the warning appears in the Serial Monitor. The correct Telegram Chat ID is also configured, but no automatic notification is received in Telegram.

This means BrokBuddy detects the low food level correctly, but it is not yet clear whether bot.sendMessage() successfully sends the message.

Fix - Check if the Telegram message is actually sent

Go back to the low-food section near the bottom of the loop() function.

Replace: bot.sendMessage(CHAT_ID, "BrokBuddy warning: Food level is low. Please refill the container.", "");

lowFoodWarningSent = true;

With:

```
bool messageSent = bot.sendMessage(
  CHAT_ID,
  "BrokBuddy warning: Food level is low. Please refill the container.",
  ""
);
if (messageSent)
{
  Serial.println("Telegram warning sent successfully");
  lowFoodWarningSent = true;
}
else
{
  Serial.println("Telegram warning failed");
}
```

```
    }

    bot_lasttime = millis();
  }

if (foodLevel < 20 && lowFoodWarningSent == false)
{
  digitalWrite(ledPin, LOW);
  Serial.println("BrokBuddy warning: Food level is low");
 bot.sendMessage(CHAT_ID, "BrokBuddy warning: Food level is low. Please refill the container.", "");
  lowFoodWarningSent = true;
}

if (foodLevel >= 20)
```

Problem - The Telegram warning fails to send

After adding the bot.sendMessage() check, the low-food detection works correctly, but the Serial Monitor shows:

Telegram warning failed

The warning is detected, but no automatic message is received in Telegram.

Cause:
The bot.sendMessage() function returns false. This means the NodeMCU attempted to send the Telegram message, but the message was not sent successfully.

Because lowFoodWarningSent is only changed to true after a successful message, it remains false. The program therefore keeps trying to send the warning again.

Result:
The low-food detection and WiFi connection are working, but the Telegram message still fails. The Telegram sending method must be corrected before the automatic warning works properly.



```
39  unsigned long bot_lasttime; // last time messages' scan has been done
40
41  const int ledPin = LED_BUILTIN;
42  int ledStatus = 0;
43  int foodLevel = 15;
44  bool lowFoodWarningSent = false;
45
46  void handleNewMessages(int numNewMessages)
47  {
```

Add: String ownerChatId = "";

```
40
41  const int ledPin = LED_BUILTIN;
42  int ledStatus = 0;
43  int foodLevel = 15;
44  bool lowFoodWarningSent = false;
45  String ownerChatId = "";
46
47  void handleNewMessages(int numNewMessages)
48  {
```

To make sure the automatic warning is sent to the correct Telegram chat, the Chat ID from an incoming message is stored in a new variable. The code below shows how ownerChatId is added and linked to the chat_id received by the bot.

```
50    Serial.println(numNewMessages);
51
52    for (int i = 0; i < numNewMessages; i++)
53    {
54      String chat_id = bot.messages[i].chat_id;
55      String text = bot.messages[i].text;
56
57      String from_name = bot.messages[i].from_name;
58      if (from_name == "")
59        from_name = "Guest";
60
61      if (text == "/ledon")
62      {
63        digitalWrite(ledPin, LOW); // turn the LED on (HIGH is the voltage level)
64        ledStatus = 1;
65        bot.sendMessage(chat_id, "Led is ON", "");
```

```
51
52    for (int i = 0; i < numNewMessages; i++)
53    {
54    String chat_id = bot.messages[i].chat_id;
55  String text = bot.messages[i].text;
56
57  ownerChatId = chat_id;
58
59  Serial.print("Owner Chat ID: ");
60  Serial.println(ownerChatId);
61
```

Make sure the void loop() function contains the updated low-food detection and Telegram notification code shown below.

```
void loop()
{
 if (millis() - bot_lasttime > BOT_MTBS)
 {
  int numNewMessages = bot.getUpdates(bot.last_message_received + 1);

  while (numNewMessages)
  {
   Serial.println("got response");
   handleNewMessages(numNewMessages);

   numNewMessages = bot.getUpdates(bot.last_message_received + 1);
  }

  bot_lasttime = millis();
 }

 if (foodLevel < 20 && lowFoodWarningSent == false && ownerChatId != "")
 {
  digitalWrite(ledPin, LOW);

  Serial.println("BrokBuddy warning: Food level is low");

  bool messageSent = bot.sendMessage(
   ownerChatId,
   "BrokBuddy warning: Food level is low. Please refill the container.",
```

```
    ""
   );

   if (messageSent)
   {
    Serial.println("Telegram warning sent successfully");
    lowFoodWarningSent = true;
   }
   else
   {
    Serial.println("Telegram warning failed");
   }
  }

  if (foodLevel >= 20)
  {
   digitalWrite(ledPin, HIGH);
   lowFoodWarningSent = false;
  }
}
```

```
void loop()
{
  if (millis() - bot_lasttime > BOT_MTBS)
  {
    int numNewMessages = bot.getUpdates(bot.last_message_received + 1);

    while (numNewMessages)
    {
      Serial.println("got response");
      handleNewMessages(numNewMessages);
      numNewMessages = bot.getUpdates(bot.last_message_received + 1);
    }

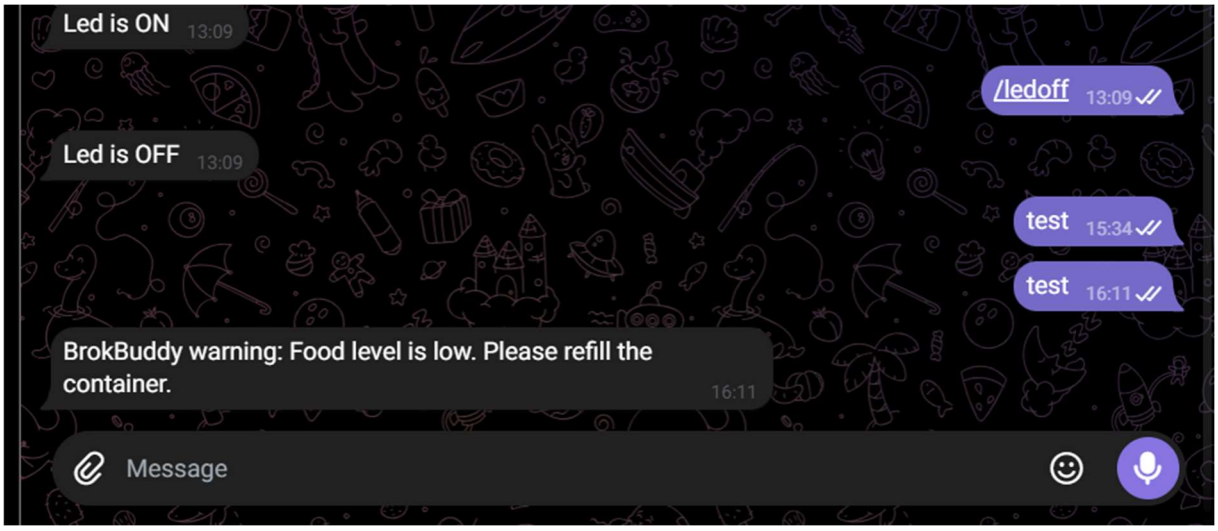
    bot_lasttime = millis();
  }

if (foodLevel < 20 && lowFoodWarningSent == false && ownerChatId != "")
{
  digitalWrite(ledPin, LOW);
  Serial.println("BrokBuddy warning: Food level is low");

  bool messageSent = bot.sendMessage(
    ownerChatId,
    "BrokBuddy warning: Food level is low. Please refill the container.",
    ""
  );

  if (messageSent)
  {
    Serial.println("Telegram warning sent successfully");
    lowFoodWarningSent = true;
  }
  else
  {
    Serial.println("Telegram warning failed");
  }
}
```

After uploading the updated code, send a message such as test to the BrokBuddy bot. This allows the NodeMCU to store the correct Telegram Chat ID. Because the simulated food level is set below 20 percent, BrokBuddy will then automatically send a low-food warning.



Result

The automatic Telegram warning is now working correctly. BrokBuddy sends a message when the simulated food level is low:

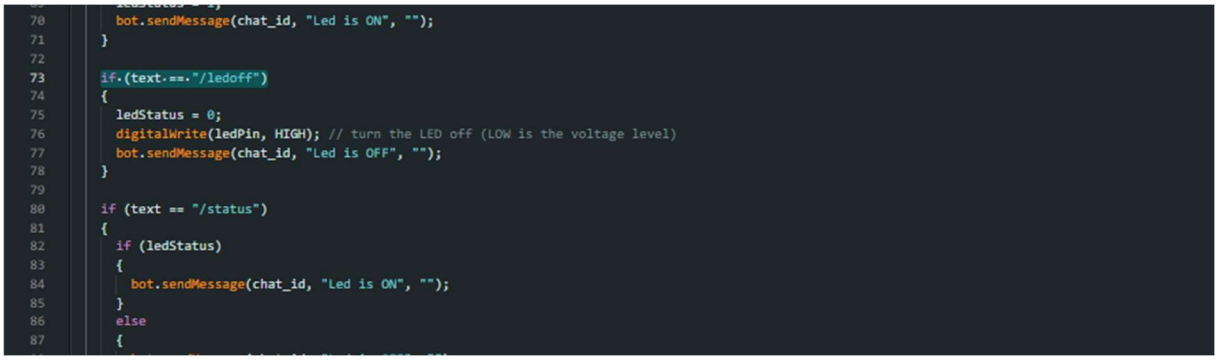
BrokBuddy warning: Food level is low. Please refill the container.

This confirms that the low-food detection, WiFi connection and Telegram notification are working together successfully.

Step 18 - Update the /status command with the food level

The current /status command only shows whether the LED is on or off. For BrokBuddy, it is more useful to also show the current food level.

Go to the handleNewMessages() function and find the current /status section:



Replace the complete /status block with:

```
if (text == "/status")
{
 String statusMessage = "BrokBuddy status\n";
 statusMessage += "Food level: ";
 statusMessage += String(foodLevel);
 statusMessage += "%\n";

 if (foodLevel < 20)
 {
  statusMessage += "Food status: LOW";
 }
 else
 {
  statusMessage += "Food status: OK";
 }

 bot.sendMessage(chat_id, statusMessage, "");
}
```

```
    }

    if (text == "/status")
  {
    String statusMessage = "BrokBuddy status\n";
    statusMessage += "Food level: ";
    statusMessage += String(foodLevel);
    statusMessage += "%\n";

    if (foodLevel < 20)
    {
      statusMessage += "Food status: LOW";
    }
    else
    {
      statusMessage += "Food status: OK";
    }

    bot.sendMessage(chat_id, statusMessage, "");
  }

    if (text == "/start")
    {
      String welcome = "Welcome to Universal Arduino Telegram Bot library, " + from_name + ".\n";
      welcome += "This is Flash Led Bot example.\n\n";
      welcome += "/ledon : to switch the Led ON\n";
```

After uploading the updated code, send the /status command to the BrokBuddy bot. The bot should return the current simulated food level and indicate whether the food status is normal or low.



Step 19 - Make the food level decrease automatically

Until now, the food level was changed manually in the Arduino code. In this step, BrokBuddy will automatically decrease the simulated food level over time. This simulates the dog eating food from the container.

First, go to the global variables near the top of the code and change:

int foodLevel = 15;

To

int foodLevel = 80;

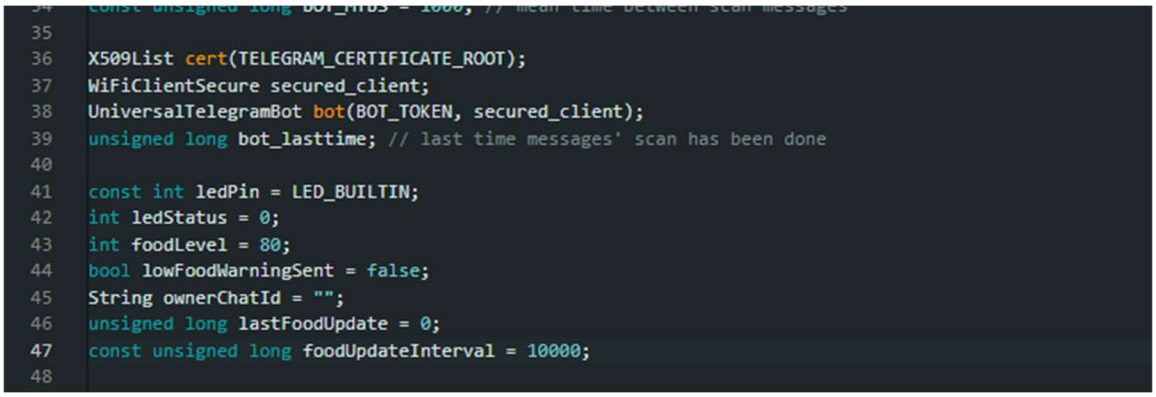
```
const int ledPin = LED_BUILTIN;
int ledStatus = 0;
int foodLevel = 80;
bool lowFoodWarningSent = false;
String ownerChatId = "";

void handleNewMessages(int numNewMessages)
{
  Serial.print("handleNewMessages ");
  Serial.println(numNewMessages);
```

Directly underneath the other global variables, add:

```
unsigned long lastFoodUpdate = 0;

const unsigned long foodUpdateInterval = 10000;
```



The value 10000 means that the simulated food level will change every 10 seconds.

Replace the complete existing void loop() function with the updated version shown below.

```
void loop()
{
 // Check new Telegram messages
 if (millis() - bot_lasttime > BOT_MTBS)
 {
  int numNewMessages = bot.getUpdates(bot.last_message_received + 1);

  while (numNewMessages)
  {
   Serial.println("got response");
   handleNewMessages(numNewMessages);
   numNewMessages = bot.getUpdates(bot.last_message_received + 1);
  }

  bot_lasttime = millis();
 }

 // Automatically decrease food level every 10 seconds
 if (millis() - lastFoodUpdate >= foodUpdateInterval && foodLevel > 0)
```

```
{
  foodLevel = foodLevel - 10;

  Serial.print("Food level: ");
  Serial.print(foodLevel);
  Serial.println("%");

  lastFoodUpdate = millis();
}

// Send warning when food level reaches 20 percent
if (foodLevel <= 20 && lowFoodWarningSent == false && ownerChatId != "")
{
  digitalWrite(ledPin, LOW);
  Serial.println("BrokBuddy warning: Food level is low");

  bool messageSent = bot.sendMessage(
    ownerChatId,
    "BrokBuddy warning: Food level is low. Please refill the container.",
    ""
  );

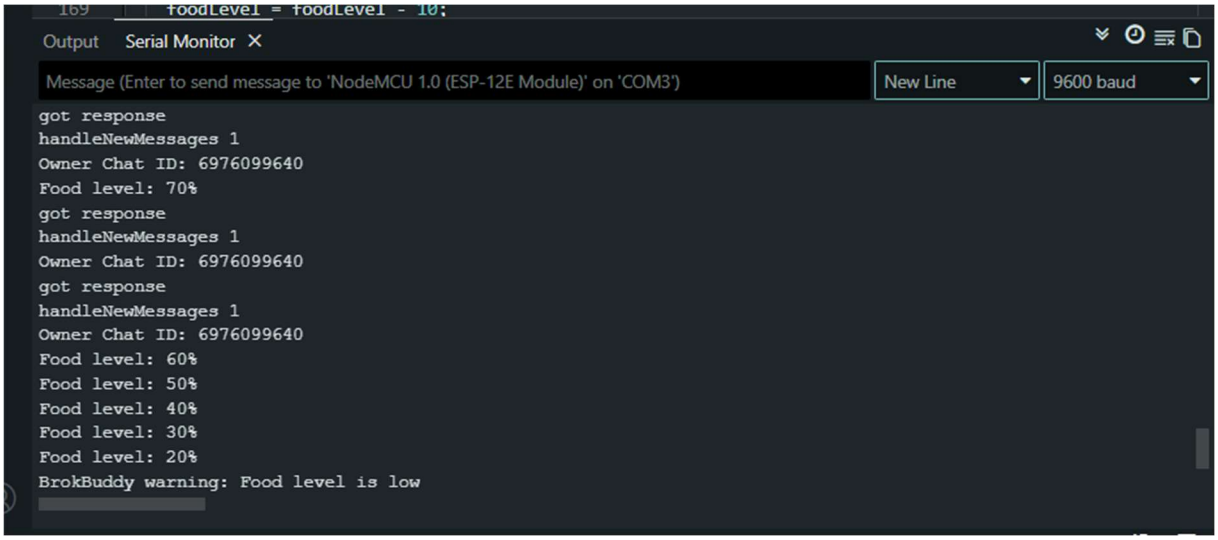
  if (messageSent)
  {
    Serial.println("Telegram warning sent successfully");
    lowFoodWarningSent = true;
  }
  else
  {
    Serial.println("Telegram warning failed");
  }
```

```
  }

  // Reset warning after refill
  if (foodLevel >= 20)
  {
    digitalWrite(ledPin, HIGH);
    lowFoodWarningSent = false;
  }
}
```



Serial monitor output:



Result

The Serial Monitor shows that the simulated food level automatically decreases from 70 percent to 20 percent.

When the food level reaches 20 percent, BrokBuddy detects that the food supply is low. At that moment, the built-in LED on the NodeMCU turns on and the Serial Monitor displays:

BrokBuddy warning: Food level is low

This confirms that the automatic food-level detection and physical warning are working correctly.

